Software Tester (QA) - Take-Home Assignment

Submission pack: Ablespace staging (staging.ablespace.io)

Please read first. I cannot open staging.ablespace.io, create an account, or run a browser from here, so I have not tested the app and cannot truthfully report real results, bugs, or screenshots. Inventing them would break the assignment rules and fall apart in your interview, where you must explain every bug and test case. This pack gives you the finished structure with the thinking done: scenarios to run, 7 fully designed test cases with expected results, filled-in templates for bugs, the compatibility grid, the summary, and a working Playwright starter. Items marked [fill in] must come from what you observe yourself. Because I have not seen the app, the screen names and field names below follow the assignment example (Create Student) and must be adjusted to what you actually see.

1. Application access

Create exactly one account at staging.ablespace.io through the normal signup flow, using a fictional name and an email you control. Record: date created [fill in], browser used [fill in]. Do not create further accounts, enter real personal data, or run security or load tests.

2. Exploratory testing: scenarios to run (8-10)

Run each, then write one line of result. Adjust wording to the real screens.

#	What to check	Result
1	Signup with valid fictional data, then log in with the new account.	[fill in]
2	Signup / login with empty fields, invalid email format, short or mismatched passwords: are the error messages clear and specific?	[fill in]
3	Login with wrong password and with unregistered email: message is helpful and does not reveal whether the email exists.	[fill in]
4	Create a student/record with all valid fields, save, then find it in the list and open it: all data saved and displayed correctly.	[fill in]
5	Create with required fields empty, then with only whitespace: validation blocks save and highlights the right fields.	[fill in]
6	Boundary text: 1 character, very long names (200+ chars), special characters and emoji (e.g. O'Brien, Zoe-Ann, accented letters): saved, displayed, and layout intact.	[fill in]
7	Double-click Save / press Enter twice quickly: no duplicate records created.	[fill in]
8	Refresh mid-form, browser Back after saving, and Forward: no data loss, no stale or duplicate state, sensible warning on unsaved changes.	[fill in]
9	Edit an existing record, save, reload: change persists; cancel edit: original data is untouched.	[fill in]
10	Search / filter: partial match, different case, no results (empty state message), clearing the search.	[fill in]

Spare ideas if you finish early: items 11-12 above (logout/session, date and number boundaries).

3. Detailed test cases

Workflow chosen: Create Student, Enter Details, Save, Search/View, Edit. Reason: it is the core data-entry flow, and errors here corrupt data every other feature relies on.

Field	Description
Test Case ID	TC-01
Title	Create student with all valid data
Preconditions	Logged in; on Create Student page
Test Data	Name: Test Student One; Email: test.one@example.com; other required fields valid
Steps	1. Open Create Student. 2. Enter valid data in all fields. 3. Click Save.
Expected Result	Success message shown; student appears in list with exactly the entered values.
Actual Result	[fill in]
Status	Pass / Fail [fill in]

Field	Description
Test Case ID	TC-02
Title	Create student with only required fields
Preconditions	Logged in
Test Data	Only mandatory fields filled; optional left empty
Steps	1. Open Create Student. 2. Fill only required fields. 3. Save. 4. Open the record.
Expected Result	Saved successfully; optional fields shown empty or "-", no errors or "undefined/null" text.
Actual Result	[fill in]
Status	Pass / Fail [fill in]

Field	Description
Test Case ID	TC-03
Title	Required field left empty
Preconditions	Logged in
Test Data	Name empty, rest valid
Steps	1. Open Create Student. 2. Leave Name blank. 3. Click Save.
Expected Result	Save blocked; inline error on Name; other entered values are kept; no record created.
Actual Result	[fill in]
Status	Pass / Fail [fill in]

Field	Description
Test Case ID	TC-04
Title	Whitespace-only and invalid email
Preconditions	Logged in
Test Data	Name: " "; Email: "test.one@"
Steps	1. Enter data. 2. Save.
Expected Result	Whitespace-only name rejected; invalid email rejected with a clear message; no record created.
Actual Result	[fill in]
Status	Pass / Fail [fill in]

Field	Description
Test Case ID	TC-05
Title	Long text and special characters
Preconditions	Logged in
Test Data	Name: 150-character string; second run: O'Brien-Smith, Zoe (accented letters)
Steps	1. Enter data. 2. Save. 3. View in list and detail page.
Expected Result	Either a clear max-length message, or saved and displayed intact without overflow, truncation bugs, or garbled characters.
Actual Result	[fill in]
Status	Pass / Fail [fill in]

Field	Description
Test Case ID	TC-06
Title	Duplicate submission (double-click Save)
Preconditions	Logged in; valid data entered
Test Data	Name: Test Student Two
Steps	1. Fill the form. 2. Double-click Save quickly. 3. Check list.
Expected Result	Exactly one record created; button disabled or guarded while saving.
Actual Result	[fill in]
Status	Pass / Fail [fill in]

Field	Description
Test Case ID	TC-07
Title	Edit student and persist after refresh
Preconditions	TC-01 record exists
Test Data	Change Name to Test Student One Edited
Steps	1. Open record. 2. Click Edit. 3. Change Name. 4. Save. 5. Refresh page. 6. Search for the new name.
Expected Result	Updated name shown everywhere after refresh; old name no longer found; cancelling an edit leaves data unchanged.
Actual Result	[fill in]
Status	Pass / Fail [fill in]

4. Bug reporting

Write up your 3 most important real bugs. Template below with a worked example of the level of detail expected (the example is illustrative only: do not submit it as a real finding).

Field	Description
Bug ID	BUG-01
Title	[Module] Specific problem, e.g. "Student record saved twice when Save is double-clicked"
Severity	Critical / High / Medium / Low (data corruption or no workaround = High+; cosmetic = Low)
Priority	High / Medium / Low
Environment	https://staging.ablespace.io, tested on [fill in] (date)
Browser / Device	e.g. Chrome 129 on Windows 11 [fill in]
Reproducibility	Always / Sometimes / Once [fill in]
Steps to Reproduce	1. Log in. 2. Go to Create Student. 3. Enter valid data. 4. Double-click Save. 5. Open student list.
Expected Result	One student record is created.

Field	Description
Actual Result	[fill in] (what really happened, e.g. two identical records)
Evidence	Screenshot / screen recording link [fill in]

Minor issues (one line each): [fill in]

5. Cross-browser and responsive report

Area / Feature	Chrome	Firefox / Edge	Mobile (390 x 844)	Notes
Signup and login	Pass / Fail	Pass / Fail	Pass / Fail	[fill in]
Main workflow	Pass / Fail	Pass / Fail	Pass / Fail	[fill in]
Forms	Pass / Fail	Pass / Fail	Pass / Fail	[fill in]
Navigation	Pass / Fail	Pass / Fail	Pass / Fail	[fill in]

Also check on iPad Mini and iPad Pro emulation (DevTools): layout breaks, text overflow, overlapping elements, unwanted horizontal scroll, hard-to-tap buttons, cut-off content. Add a note for every Fail.

6. Test summary (template, about half a page)

Testing performed	I explored signup/login and the Create/Search/Edit Student workflow on Chrome [fill in], Firefox/Edge [fill in], the mobile app [fill in], and iPad Mini/Pro emulation. I ran [fill in] scenarios and [fill in] detailed test cases.
Issues found	[fill in] bugs in total (BUG-01 to BUG-0x). I would fix BUG-[fill in] first because [it loses/corrupts data, blocks a core flow, or affects most users].
Areas not tested	Security, penetration and load testing (prohibited by the rules); creating multiple accounts (prohibited); [fill in] (anything else skipped, with reason).
What I would test next	Given 1-2 more hours: (1) permissions and role-based access between user types; (2) data persistence across sessions and devices; (3) file upload/import features if present; (4) accessibility basics (keyboard navigation, labels, contrast); (5) slow-network and offline behaviour. These matter because they are where real users most often hit unexpected failures.

7. Optional: Playwright starter (TypeScript)

Flow: Login, open the student list, verify the page loads. Locators use roles and labels, so adjust names to match the real UI. Credentials come from environment variables.

tests/login.spec.ts

```
import { test, expect } from '@playwright/test';
import 'dotenv/config';

const BASE_URL = process.env.BASE_URL ?? 'https://staging.ablespace.io';
const EMAIL = process.env.QA_EMAIL;
const PASSWORD = process.env.QA_PASSWORD;

test.describe('Login flow', () => {
  test.skip(!EMAIL || !PASSWORD, 'Set QA_EMAIL and QA_PASSWORD');

  test('registered user can log in and reach the dashboard', async ({ page }) => {
    await page.goto(BASE_URL);
    await page.getByRole('link', { name: /log ?in/i }).click();

    await page.getByLabel(/email/i).fill(EMAIL!);
    await page.getByLabel(/password/i).fill(PASSWORD!);
    await page.getByRole('button', { name: /log ?in|sign in/i }).click();

    await expect(page).not.toHaveURL(/login/);
    await expect(page.getByRole('navigation')).toBeVisible();
  });

  test('wrong password shows an error and stays on login', async ({ page }) => {
    await page.goto(BASE_URL);
    await page.getByRole('link', { name: /log ?in/i }).click();
    await page.getByLabel(/email/i).fill(EMAIL!);
    await page.getByLabel(/password/i).fill('wrong-password-123');
    await page.getByRole('button', { name: /log ?in|sign in/i }).click();

    await expect(page.getByRole('alert')).toBeVisible();
  });
});
```

README.md

```
# Ablespace Playwright test

## Install
npm install
npx playwright install

## Credentials (never commit these)
Create a .env file in the project root:
  QA_EMAIL=your_test_account_email
  QA_PASSWORD=your_test_account_password
.env is listed in .gitignore.

## Run
npx playwright test
npx playwright show-report

## Notes
- Uses the account created manually; signup is not automated.
- Locators use roles/labels; update if the UI text differs.
```

.gitignore: .env, node_modules/, playwright-report/, test-results/

Dependencies: npm i -D @playwright/test dotenv typescript